AI/ML FRAMEWORKS

Hugging Face Transformers

From one-line pipeline() inference to LoRA fine-tuning — the complete working guide to the Hugging Face Transformers ecosystem.

transformers · huggingface · NLP · LLM · fine-tuning · LoRA · tokenization

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What It Is and Why It Matters
2. 2. Setup and Getting Started
3. 3. The pipeline() — Zero-to-Inference
4. 4. AutoTokenizer and AutoModel — The Manual API
5. 5. Tokenization in Depth
6. 6. Text Generation
7. 7. Embeddings and Feature Extraction
8. 8. The datasets Library
9. 9. Fine-Tuning with Trainer
10. 10. PEFT and LoRA
11. 11. Saving, Loading, and Sharing
12. 12. Cheat-Sheet Reference
13. 13. Best Practices and Common Pitfalls
14. 14. Interview Questions & Answers
15. 15. Further Learning

Hugging Face **Transformers** is the de-facto standard library for working with pretrained deep-learning models — from a one-line `pipeline()` that classifies text, to full fine-tuning of a 7B-parameter LLM with LoRA. It gives you a unified Python API over tens of thousands of models on the Hub, backed interchangeably by PyTorch, and ships the tokenizers, training loop, and inference utilities you need to go from prototype to production.

1. What It Is and Why It Matters

Before Transformers, using a state-of-the-art NLP model meant cloning a research repo, reconciling framework versions, and reimplementing preprocessing. The library collapses that into three composable abstractions you will use everywhere: a **tokenizer** that turns text into model-ready integer tensors, a **model** that maps those tensors to hidden states or predictions, and a **pipeline** that wires both together for a task.

THE HUB

A git-backed registry of 1M+ models, datasets, and demos. Any checkpoint is one string away: `"bert-base-uncased"`, `"meta-llama/Llama-3.1-8B"`.

AUTO CLASSES

`AutoModel` / `AutoTokenizer` read a model's config and instantiate the correct architecture — your code stays identical across BERT, GPT, T5, or Llama.

ONE ECOSYSTEM

`transformers`, `datasets`, `tokenizers`, `accelerate`, `peft`, and `evaluate` interoperate so training and inference share one data path.

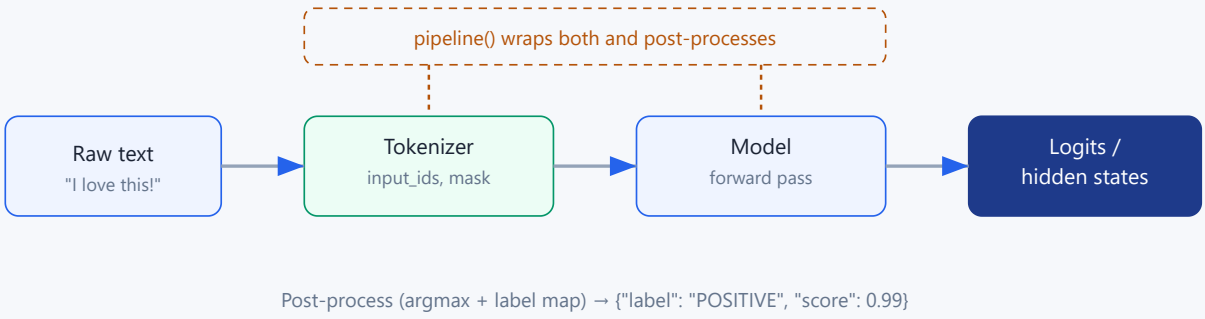


Fig 1 — The core data path. `pipeline()` is a convenience wrapper over the same tokenizer + model you can also drive manually.

> **Mental model**
> Think of Transformers as "drivers for neural networks". The model file is the hardware; the `config.json` tells the Auto classes which driver to load; the tokenizer is the cable that converts your text into the exact signal the model was trained on.

2. Setup and Getting Started

Install the library with a backend. PyTorch is the default and most widely supported. Add the sibling libraries you'll need for data and efficient fine-tuning.

BASH

```bash
# Isolated environment
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate

# Core + PyTorch backend
pip install "transformers[torch]"

# Common companions
pip install datasets           # streaming/efficient dataset loading
pip install accelerate         # device placement + distributed training
pip install evaluate           # metrics (accuracy, f1, bleu, ...)
pip install peft               # LoRA / parameter-efficient fine-tuning
pip install sentencepiece      # tokenizers for T5, Llama, etc.

# Verify
python -c "import transformers; print(transformers.__version__)"
```

BASH

```bash
# Authenticate to download gated models (e.g. Llama) and push your own
huggingface-cli login          # paste a token from huggingface.co/settings/tokens

# Caching: downloaded weights live here (override with HF_HOME)
export HF_HOME="$HOME/.cache/huggingface"
```

Tip

Set `HF_HUB_ENABLE_HF_TRANSFER=1` and `pip install hf_transfer` for much faster multi-gigabyte downloads. In notebooks, use `from huggingface_hub import notebook_login`.

3. The pipeline() — Zero-to-Inference

`pipeline()` is the fastest path to a working result. You name a task; it picks a sensible default model, downloads it, and handles pre/post-processing. Pass an explicit `model=` for reproducibility in real code.

PYTHON

```python
from transformers import pipeline

# Sentiment analysis (default model chosen for the task)
clf = pipeline("sentiment-analysis")
print(clf("Hugging Face makes NLP wonderfully simple."))
# [{'label': 'POSITIVE', 'score': 0.9998}]

# Pin a model explicitly + run on GPU (device=0) or Apple Silicon ("mps")
clf = pipeline(
    task="sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english",
    device=0,                       # -1 = CPU
)

# Batch many inputs at once — far faster than a Python loop
results = clf(["Great value.", "Terrible support.", "It's fine."])
```

The task string selects the output shape and post-processing. A representative set:

Task string	What it does	Typical default head
`sentiment-analysis` / `text-classification`	Label a whole sequence	SequenceClassification
`ner` / `token-classification`	Tag each token (entities)	TokenClassification
`question-answering`	Extract an answer span from a context	QuestionAnswering
`fill-mask`	Predict masked tokens	MaskedLM
`summarization` / `translation`	Seq2seq text generation	Seq2SeqLM
`text-generation`	Autoregressive completion / chat	CausalLM
`feature-extraction`	Return raw hidden states (embeddings)	base Model
`zero-shot-classification`	Classify into arbitrary labels via NLI	SequenceClassification
`automatic-speech-recognition`	Transcribe audio (Whisper)	SpeechSeq2Seq

⚠ **Reuse the pipeline**
Constructing a pipeline reloads weights from disk every time. Create it *once* at startup and reuse it; never build one inside a request handler or a loop.

4. AutoTokenizer and AutoModel — The Manual API

When you need control — custom pre/post-processing, access to hidden states, or integration into a training loop — drop below the pipeline. The `Auto*` classes inspect `config.json` and return the right concrete class.

PYTHON

```python
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

ckpt = "distilbert-base-uncased-finetuned-sst-2-english"
tok = AutoTokenizer.from_pretrained(ckpt)
model = AutoModelForSequenceClassification.from_pretrained(ckpt)

# Tokenize → PyTorch tensors, padded + truncated to the model's limit
inputs = tok(
    ["I love this", "I hate waiting"],
    padding=True, truncation=True, return_tensors="pt",
)
# inputs = {'input_ids': tensor([ ... ]), 'attention_mask': tensor([ ... ])}

with torch.no_grad():                  # inference: no gradients needed
    logits = model(**inputs).logits    # shape [batch, num_labels]

probs = torch.softmax(logits, dim=-1)
preds = probs.argmax(dim=-1)
# Map ids to names via the model's config
labels = [model.config.id2label[i.item()] for i in preds]
print(labels)                          # ['POSITIVE', 'NEGATIVE']
```

The AutoModel family

Pick the class that matches your task — the suffix determines which *head* is attached on top of the base encoder/decoder.

Class	Use for	Output
`AutoModel`	Raw embeddings / custom heads	`last_hidden_state`
`AutoModelForSequenceClassification`	Sentiment, topic, NLI	logits [batch, labels]
`AutoModelForTokenClassification`	NER, POS tagging	logits [batch, seq, labels]
`AutoModelForQuestionAnswering`	Extractive QA	start/end logits
`AutoModelForCausalLM`	GPT-style generation, chat LLMs	next-token logits
`AutoModelForSeq2SeqLM`	Translation, summarization (T5, BART)	decoder logits
`AutoModelForMaskedLM`	Masked-token prediction (BERT)	vocab logits

5. Tokenization in Depth

Tokenization is where most subtle bugs hide. Modern tokenizers use **subword** algorithms (WordPiece, Byte-Pair Encoding, SentencePiece/Unigram) so any string — including unseen words and emoji — maps to a finite vocabulary. The tokenizer also adds special tokens the model expects and produces an `attention_mask` marking real vs. padded positions.

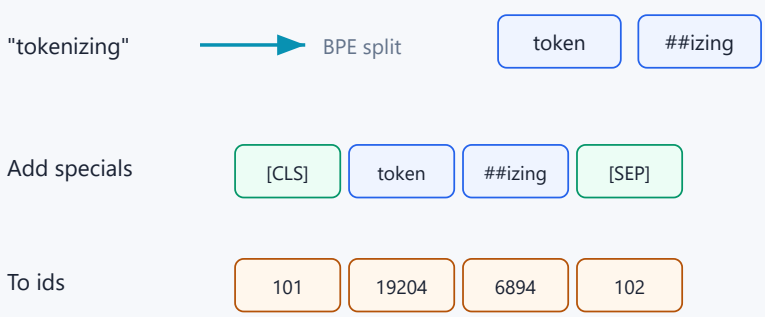


Fig 2 — Tokenization stages: subword split → add special tokens → map to vocabulary ids. The exact split and specials are baked into each checkpoint.

PYTHON

```python
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("bert-base-uncased")

enc = tok("Tokenizing isn't hard.")
print(enc["input_ids"])            # [101, 19204, 6026, 2003, 2025, 2524, 1012, 102]
print(tok.convert_ids_to_tokens(enc["input_ids"]))
# ['[CLS]', 'token', '##izing', 'is', "n't", 'hard', '.', '[SEP]']

# Round-trip back to text
print(tok.decode(enc["input_ids"], skip_special_tokens=True))  # "tokenizing isn't har

# Key arguments you will use constantly
batch = tok(
    ["short", "a much longer sequence that may exceed the limit"],
    padding="longest",       # or "max_length"
    truncation=True,
    max_length=128,
    return_tensors="pt",     # "pt" torch, "np" numpy, None lists
)
```

Chat templates

Instruction-tuned LLMs expect a specific role-formatted prompt. Never hand-format it — use the tokenizer's built-in template so special tokens match training exactly.

PYTHON

```python
messages = [
    {"role": "system", "content": "You are a concise assistant."},
    {"role": "user", "content": "Explain attention in one sentence."},
]
prompt = tok.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,   # append the assistant turn marker
)
```

Golden rule

Always load the tokenizer from the *same checkpoint* as the model. A model's weights are meaningless if the token ids, special tokens, or vocabulary don't match what it was trained on.

6. Text Generation

Causal LMs generate autoregressively: predict the next token, append it, repeat. The `generate()` method exposes decoding strategy, length, and sampling controls. Understanding these knobs is essential interview material.

PYTHON

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

ckpt = "Qwen/Qwen2.5-0.5B-Instruct"
tok = AutoTokenizer.from_pretrained(ckpt)
model = AutoModelForCausalLM.from_pretrained(ckpt, torch_dtype="auto", device_map="aut

messages = [{"role": "user", "content": "Give me a haiku about tensors."}]
inputs = tok.apply_chat_template(
    messages, add_generation_prompt=True, return_tensors="pt"
).to(model.device)

out = model.generate(
    inputs,
    max_new_tokens=64,       # cap generated length (NOT total length)
    do_sample=True,          # False = greedy/deterministic
    temperature=0.7,         # <1 sharper, >1 more random
    top_p=0.9,               # nucleus sampling
    top_k=50,
    repetition_penalty=1.1,
)
# Slice off the prompt tokens before decoding
text = tok.decode(out[0][inputs.shape[-1]:], skip_special_tokens=True)
print(text)
```

Strategy	Settings	When to use
Greedy	`do_sample=False`	Deterministic, factual, short answers
Beam search	`num_beams=4`	Translation/summarization quality
Sampling	`do_sample=True, temperature, top_p`	Creative, diverse open-ended text

✦ **Streaming + speed**

Pass a `TextIteratorStreamer` to `generate()` to yield tokens as they arrive. Set `model.generation_config.pad_token_id = tok.eos_token_id` to silence padding warnings, and prefer `torch_dtype=torch.bfloat16` on modern GPUs to halve memory.

7. Embeddings and Feature Extraction

For semantic search, clustering, or RAG retrieval you need a fixed-size vector per text. Transformers gives raw hidden states; you pool them into a sentence embedding. **Mean pooling** over the attention mask is the standard recipe for most sentence encoders.

PYTHON

```python
from transformers import AutoTokenizer, AutoModel
import torch
import torch.nn.functional as F

ckpt = "sentence-transformers/all-MiniLM-L6-v2"
tok = AutoTokenizer.from_pretrained(ckpt)
model = AutoModel.from_pretrained(ckpt)

def mean_pool(last_hidden, mask):
    mask = mask.unsqueeze(-1).float()            # [batch, seq, 1]
    summed = (last_hidden * mask).sum(1)
    counts = mask.sum(1).clamp(min=1e-9)
    return summed / counts                       # [batch, hidden]

sentences = ["A dog runs in the park.", "A canine sprints across a field."]
enc = tok(sentences, padding=True, truncation=True, return_tensors="pt")
with torch.no_grad():
    out = model(**enc)

emb = mean_pool(out.last_hidden_state, enc["attention_mask"])
emb = F.normalize(emb, p=2, dim=1)               # unit vectors for cosine sim

similarity = (emb[0] @ emb[1]).item()            # dot of normalized == cosine
print(f"cosine similarity: {similarity:.3f}")    # ~0.7 (semantically close)
```

> **Shortcut**
>
> The `sentence-transformers` library wraps this pooling for you: `SentenceTransformer(ckpt).encode(sentences)`. Use raw Transformers when you need custom pooling or want zero extra dependencies.

8. The datasets Library

The `datasets` library loads data lazily via Apache Arrow (memory-mapped, so it won't blow up your RAM), and offers fast `.map()` transforms that integrate directly with the `Trainer`.

PYTHON

```python
from datasets import load_dataset

# Load from the Hub (train/validation/test splits come as a DatasetDict)
ds = load_dataset("imdb")
print(ds)
# DatasetDict({ train: Dataset(25000), test: Dataset(25000) })

# Load your own files
local = load_dataset("csv", data_files={"train": "train.csv", "test": "test.csv"})

# Tokenize the whole dataset in batches (cached to disk automatically)
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def preprocess(batch):
    return tok(batch["text"], truncation=True, max_length=256)

ds = ds.map(preprocess, batched=True)
ds = ds.remove_columns(["text"]).rename_column("label", "labels")
ds.set_format("torch")

# Stream a huge dataset without downloading it all
big = load_dataset("c4", "en", split="train", streaming=True)
for example in big.take(2):
    print(example["text"][:80])
```

9. Fine-Tuning with Trainer

The `Trainer` class implements a robust training loop — mixed precision, gradient accumulation, checkpointing, logging, distributed training — so you supply only the model, data, and `TrainingArguments`. Here is a complete, runnable classification fine-tune.

PYTHON

```python
import numpy as np
import evaluate
from datasets import load_dataset
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer, DataCollatorWithPadding,
)

ckpt = "distilbert-base-uncased"
tok = AutoTokenizer.from_pretrained(ckpt)

ds = load_dataset("imdb")
def prep(b): return tok(b["text"], truncation=True, max_length=256)
ds = ds.map(prep, batched=True)

# Dynamic padding per-batch is faster than padding everything to max_length
collator = DataCollatorWithPadding(tokenizer=tok)

model = AutoModelForSequenceClassification.from_pretrained(
    ckpt, num_labels=2,
    id2label={0: "NEG", 1: "POS"}, label2id={"NEG": 0, "POS": 1},
)

accuracy = evaluate.load("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return accuracy.compute(predictions=preds, references=labels)

args = TrainingArguments(
    output_dir="imdb-distilbert",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",       # evaluate each epoch
    save_strategy="epoch",
    load_best_model_at_end=True,
    fp16=True,                   # mixed precision on GPU
    push_to_hub=False,
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds["train"].shuffle(seed=42).select(range(5000)),
    eval_dataset=ds["test"].select(range(1000)),
```

```
    processing_class=tok,        # (was tokenizer=) handles saving the tokenizer
    data_collator=collator,
    compute_metrics=compute_metrics,
)

trainer.train()
trainer.save_model("imdb-distilbert/best")   # saves weights + tokenizer + config
```

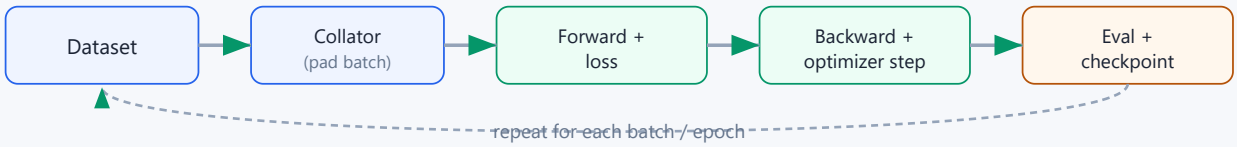


Fig 3 — What Trainer orchestrates per step: batch → pad → forward/loss → backward/step, with periodic eval and checkpointing.

10. PEFT and LoRA

Full fine-tuning of a billion-parameter model updates every weight — enormous memory and storage. **Parameter-Efficient Fine-Tuning (PEFT)** freezes the base model and trains a tiny number of extra parameters. The dominant method, **LoRA (Low-Rank Adaptation)**, injects two small matrices *A* (d×r) and *B* (r×d) whose product approximates the weight update, where rank *r* is typically 8-64. You train <1% of the parameters and ship a few-MB adapter instead of a multi-GB checkpoint.

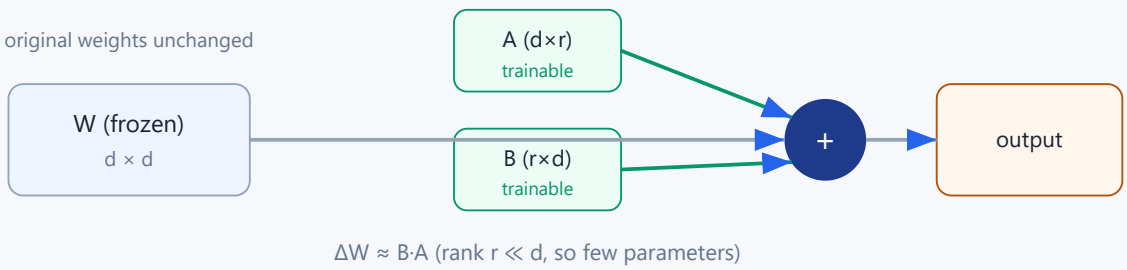


Fig 4 — LoRA. The frozen weight W is kept; only the low-rank adapters A and B are trained, and their product is added to W's output.

PYTHON

```python
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM

base = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-0.5B", torch_dtype="auto", device_map="auto",
)

lora = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                    # rank
    lora_alpha=32,           # scaling (effective LR multiplier ~ alpha/r)
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],   # which linear layers get adapters
)

model = get_peft_model(base, lora)
model.print_trainable_parameters()
# trainable params: 1.08M || all params: 495M || trainable%: 0.22

# Train with the SAME Trainer as before, then save just the adapter (a few MB)
model.save_pretrained("qwen-lora-adapter")

# Later: reload base + adapter for inference
from peft import PeftModel
base = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-0.5B")
model = PeftModel.from_pretrained(base, "qwen-lora-adapter")
```

✦ **QLoRA**

Combine LoRA with a 4-bit quantized base (`bitsandbytes` , `load_in_4bit=True`) to fine-tune a 7B model on a single consumer GPU. The base is frozen anyway, so quantizing it costs little accuracy.

11. Saving, Loading, and Sharing

PYTHON

```python
# Save locally — produces config.json, model weights (safetensors), tokenizer files
model.save_pretrained("my-model")
tok.save_pretrained("my-model")

# Reload later
model = AutoModelForSequenceClassification.from_pretrained("my-model")

# Push to the Hub (requires huggingface-cli login)
model.push_to_hub("your-username/my-model")
tok.push_to_hub("your-username/my-model")
```

safetensors

The default weight format is `.safetensors` — a safe, fast, zero-copy format that cannot execute arbitrary code on load (unlike pickle-based `.bin`). Prefer it, especially for third-party checkpoints.

12. Cheat-Sheet Reference

Goal	Call
Quick inference	`pipeline(task, model= ... )`
Load tokenizer	`AutoTokenizer.from_pretrained(ckpt)`
Load model + head	`AutoModelForXxx.from_pretrained(ckpt)`
Encode text	`tok(text, padding=True, truncation=True, return_tensors="pt")`
Decode ids	`tok.decode(ids, skip_special_tokens=True)`
Chat prompt	`tok.apply_chat_template(msgs, add_generation_prompt=True)`
Generate text	`model.generate(ids, max_new_tokens= ... )`
Load data	`load_dataset(name_or_format, ... )`
Map transform	`ds.map(fn, batched=True)`
Train	`Trainer(model, args, ... ).train()`
Add LoRA	`get_peft_model(model, LoraConfig( ... ))`
Save / share	`model.save_pretrained(dir)` / `.push_to_hub(repo)`

Model families at a glance

Family	Type	Best for
BERT / RoBERTa / DeBERTa	Encoder-only	Classification, NER, embeddings
GPT / Llama / Qwen / Mistral	Decoder-only	Generation, chat, few-shot
T5 / BART / mBART	Encoder-decoder	Summarization, translation
Whisper	Speech seq2seq	Transcription / ASR
CLIP / ViT	Vision / multimodal	Image classification, retrieval

13. Best Practices and Common Pitfalls

DO

- Pin exact checkpoint strings and a `revision=` commit hash for reproducibility.
- Use `torch.no_grad()` or `@torch.inference_mode()` for inference.
- Batch inputs; use `DataCollatorWithPadding` for dynamic padding.
- Prefer `bfloat16` / `fp16` on GPU to cut memory and speed up.

AVOID

- Building a pipeline inside a request handler or loop.
- Mismatched tokenizer and model checkpoints.
- Forgetting to slice the prompt off generated output before decoding.
- Padding everything to `max_length` when dynamic padding is faster.

✓ **Best practice**

Keep model loading separate from inference. Load once (ideally at process start), move to the right device, set `model.eval()`, and reuse the handle. This dominates latency in real services.

⚠ **Pitfall: OOM during training**

If you hit CUDA out-of-memory, reduce `per_device_train_batch_size` and compensate with `gradient_accumulation_steps`, enable `gradient_checkpointing=True`, and switch to `fp16` / `bf16`. These trade compute/time for memory.

✕ **Pitfall: trusting remote code**

Some models require `trust_remote_code=True`, which executes code from the repo on your machine. Only enable it for sources you trust, and pin a specific `revision`.

14. Interview Questions & Answers

> **What do the Auto classes actually do?**

They read the model's `config.json` (specifically `architectures` / `model_type`) and dynamically instantiate the matching concrete class — e.g. `AutoModelForCausalLM` on a Llama config returns `LlamaForCausalLM`. This lets one code path work across architectures and keeps your code decoupled from a specific model.

Why must the tokenizer match the model?

The model learned a mapping from specific token ids and special tokens to meaning. A different tokenizer produces different ids, vocabulary, and special-token placement, so the inputs no longer mean what the model expects — outputs become garbage. Always load both from the same checkpoint.

What is the attention_mask for?

When batching sequences of different lengths you pad shorter ones to a common length. The attention mask is 1 for real tokens and 0 for padding, telling the attention mechanism to ignore padded positions so they don't affect the output.

Explain temperature, top_k, and top_p in generation.

They shape the next-token distribution during sampling. Temperature scales logits before softmax (<1 sharpens toward the top token, >1 flattens for more randomness). top_k keeps only the k highest-probability tokens. top_p (nucleus) keeps the smallest set of tokens whose cumulative probability reaches p. Greedy decoding (`do_sample=False`) always takes the argmax.

How does LoRA reduce fine-tuning cost?

It freezes the pretrained weights and learns a low-rank update $\Delta W = B \cdot A$, where A is $d \times r$ and B is $r \times d$ with rank $r \ll d$. Only A and B are trained — often <1% of parameters — so optimizer state and gradients are tiny, and the resulting adapter is a few MB. The base model is shared across many task adapters.

What problem does the datasets library's Arrow backend solve?

It memory-maps data from disk instead of loading it all into RAM, so you can work with datasets larger than memory. It also enables fast columnar access, zero-copy slicing, on-disk caching of `.map()` results, and streaming mode for datasets too large to download.

When would you choose full fine-tuning over LoRA?

When you have ample compute, the target task differs substantially from pretraining, and you need maximum accuracy with no inference-time adapter overhead. For most practical adaptation — limited GPU, many tasks, fast iteration — LoRA/QLoRA gives comparable quality at a fraction of the cost.

How do you get a sentence embedding from a BERT-style model?

> Run the base model to get `last_hidden_state` (one vector per token), then pool. Mean pooling over non-padding tokens (weighted by the attention mask) is the standard; some models use the `[CLS]` token. Normalize to unit length so dot product equals cosine similarity.

> **Encoder-only vs decoder-only vs encoder-decoder — when to use each?**
>
> Encoder-only (BERT) sees the whole sequence bidirectionally — best for understanding tasks like classification and NER. Decoder-only (GPT, Llama) is causal/autoregressive — best for generation and chat. Encoder-decoder (T5, BART) encodes an input then generates a transformed output — best for translation and summarization.

> **What does gradient accumulation do and why use it?**
>
> It runs several forward/backward passes, summing gradients, before a single optimizer step. This simulates a larger effective batch size (small batch × accumulation steps) without the memory of a big batch — essential for training large models on limited VRAM.

> **Why prefer safetensors over pickle checkpoints?**
>
> Pickle (`.bin`) can execute arbitrary code when loaded, a security risk for untrusted models. safetensors stores only tensor data, loads faster via zero-copy/mmap, and cannot run code — making it the safe default.

> **How would you serve a fine-tuned model efficiently in production?**
>
> Load the model once at startup in eval mode on the GPU with bf16; batch incoming requests; use `torch.inference_mode()`; cache the tokenizer. For high-throughput LLM generation, use a dedicated serving engine (e.g. TGI or vLLM) that adds KV-cache management, continuous batching, and paged attention rather than calling `generate()` per request.

15. Further Learning

- **Official course** — the free Hugging Face LLM/NLP course walks through tokenizers, fine-tuning, and the Trainer end to end.
- **Docs deep-dives** — read the `generate()` and `GenerationConfig` reference, and the PEFT guide for LoRA/QLoRA/prefix tuning.
- **accelerate** — learn multi-GPU and mixed-precision training to scale beyond one device.
- **sentence-transformers** — for production embedding and semantic search.
- **Serving** — explore Text Generation Inference (TGI) and vLLM for low-latency, high-throughput LLM serving.
- **Build something** — fine-tune a small model on your own labelled data with LoRA, push the adapter to the Hub, and wire it into a RAG or chat demo.